

Projeto — Cidades ESGInteligentes

Eco Compliance API: CI/CD, containerização e deploy em staging e produção

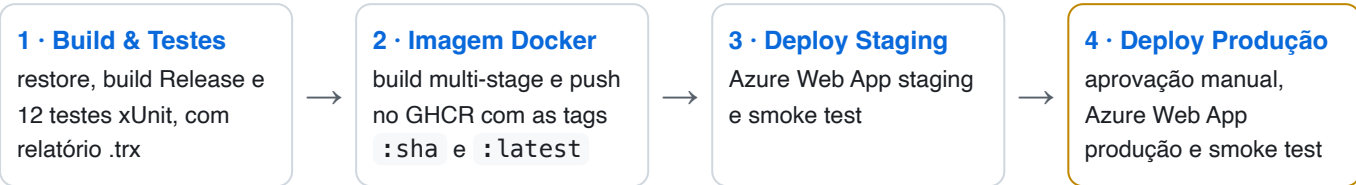
Integrante	Luiz Alberto Silva de Santana (RM565199)
Aplicação	API REST em C# / ASP.NET Core 8 para compliance ambiental (ESG)
Repositório	github.com/Daikan1/eco-compliance-dotnet
Pipeline	GitHub Actions: build, testes, imagem Docker, staging e produção
Staging	eco-compliance-rm565199-staging.azurewebsites.net
Produção	eco-compliance-rm565199-production.azurewebsites.net

Sumário

1. Descrição do pipeline: ferramenta, etapas e lógica
2. Docker: arquitetura, comandos e imagem criada
3. Prints do pipeline rodando (build, testes e deploy)
4. Prints dos ambientes staging e produção funcionando
5. Desafios encontrados e como foram resolvidos
6. Checklist de entrega

1. Descrição do pipeline

Ferramenta: GitHub Actions (`.github/workflows/ci-cd.yml`). O pipeline é disparado a cada *push* na branch `main` , em *pull requests* e manualmente (`workflow_dispatch`). A imagem vai para o **GitHub Container Registry** (GHCR), e o deploy é feito no **Azure App Service** (Web App for Containers, Linux), com um Web App por ambiente.



Etapa	O que faz
Build & Testes	<code>dotnet restore</code> , <code>dotnet build -c Release</code> e <code>dotnet test</code> com logger TRX. Os resultados são publicados como artefato (<code>test-results</code>). Se algum teste falhar, o pipeline para aqui. Em <i>pull requests</i> , só esta etapa roda.
Imagem Docker (GHCR)	Faz o build com o Dockerfile multi-stage, usando o cache de camadas do Actions. O SHA do commit entra na imagem como <code>APP_VERSION</code> , e a imagem é publicada em <code>ghcr.io/daikan1/eco-compliance-api</code> .
Deploy Staging	<code>azure/webapps-deploy@v3</code> aponta o Web App de staging para a imagem <code>:<sha></code> . O smoke test consulta <code>GET /</code> até a resposta trazer o SHA novo e valida <code>GET /health</code> . Se a versão não subir em 5 minutos, o job falha.
Deploy Produção	Só roda se staging passou. O GitHub Environment <code>production</code> exige aprovação manual de um revisor. A mesma imagem validada em staging é promovida, e o smoke test se repete.

Lógica e boas práticas

Imagem imutável	O artefato é construído uma única vez. Staging e produção recebem exatamente a mesma imagem <code>:<sha></code> ; só mudam as variáveis de ambiente.
Promoção com gate	O deploy em produção depende do sucesso de staging e de aprovação humana.
Segredos fora do código	Cada GitHub Environment guarda <code>AZURE_WEBAPP_PUBLISH_PROFILE</code> (secret) e <code>AZURE_WEBAPP_NAME</code> (variável). A connection string, a senha de admin e a chave JWT ficam nas <i>App Settings</i> de cada Web App.
Rastreabilidade	<code>GET /</code> informa o ambiente e o commit em execução, o que permite verificar qual versão está em cada ambiente.
Infraestrutura como script	<code>scripts/azure-setup.sh</code> provisiona Resource Group, App Service Plan (B1, East US 2), os dois Web Apps, as App Settings e os GitHub Environments.

2. Docker: arquitetura, comandos e imagem

Arquitetura

Local: Docker Compose

rede eco-net (bridge) volume oracle-data .env

api (build do Dockerfile, porta 8080): sobe só depois que o Oracle está *healthy* e aplica as migrations do EF Core automaticamente.

oracle (gvenzl/oracle-free:23-slim-faststart , porta 1521): Oracle 23ai Free com health check e dados persistidos no volume.

Nuvem: Azure App Service

App Service Plan B1 Linux East US 2

eco-compliance-rm565199-staging
(ASPNETCORE_ENVIRONMENT=Staging)

eco-compliance-rm565199-production
(ASPNETCORE_ENVIRONMENT=Production)

Os dois executam a imagem do GHCR e se conectam ao Oracle da FIAP (oracle.fiap.com.br).

Dockerfile (multi-stage)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["EcoCompliance.API/EcoCompliance.API.csproj", "EcoCompliance.API/"]
RUN dotnet restore "EcoCompliance.API/EcoCompliance.API.csproj" # camada cacheada
COPY . .
WORKDIR "/src/EcoCompliance.API"
RUN dotnet publish "EcoCompliance.API.csproj" \
    -c Release -o /app/publish /p:UseAppHost=false --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
ARG APP_VERSION=local
WORKDIR /app
EXPOSE 8080
ENV ASPNETCORE_ENVIRONMENT=Production ASPNETCORE_HTTP_PORTS=8080 APP_VERSION=${APP_VERSION}
COPY --from=build /app/publish .
USER $APP_UID # sem root
ENTRYPOINT ["dotnet", "EcoCompliance.API.dll"]
```

Estratégias

- Multi-stage: o SDK fica fora da imagem final, que leva só o runtime ASP.NET
- Restore em camada separada, para acelerar os rebuilds
- Execução como usuário não-root (\$APP_UID)
- .dockerignore exclui .env , appsettings.Development.json , bin/obj e docs
- Credenciais entram apenas por variáveis de ambiente

Imagem criada

Repositório	ghcr.io/daikan1/eco-compliance-api
Tags	latest , cf119f524c45...
Base	mcr.microsoft.com/dotnet/aspnet:8.0
Tamanho	≈ 94 MB compactada
Plataforma	linux/amd64 (build no runner ubuntu do GitHub)

Comandos utilizados

```
cp .env.example .env # variáveis do ambiente local
docker compose up -d --build # build da imagem + sobe Oracle e API
docker compose ps # status dos serviços (oracle healthy, api up)
docker compose logs -f api # acompanha migrations e startup
docker build -t eco-compliance-api . # build avulso da imagem
docker pull ghcr.io/daikan1/eco-compliance-api:latest
docker compose down [-v] # derruba (e remove o volume do banco)
```

3. Prints do pipeline rodando

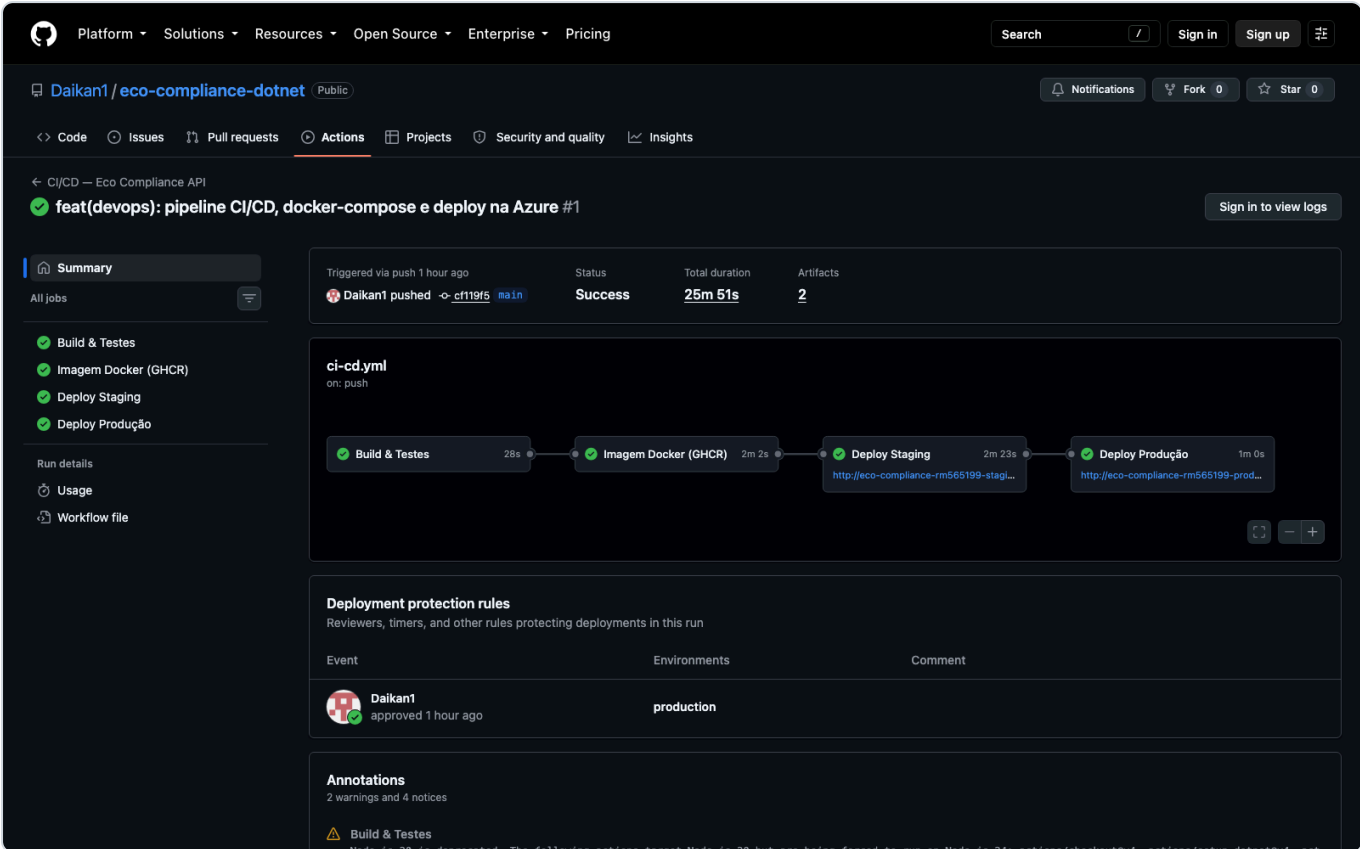


Figura 1: Run #1 (commit cf119f5). Os 4 jobs concluíram com sucesso em sequência, e o registro "Deployment protection rules" mostra a aprovação manual de **production** por Daikan1.

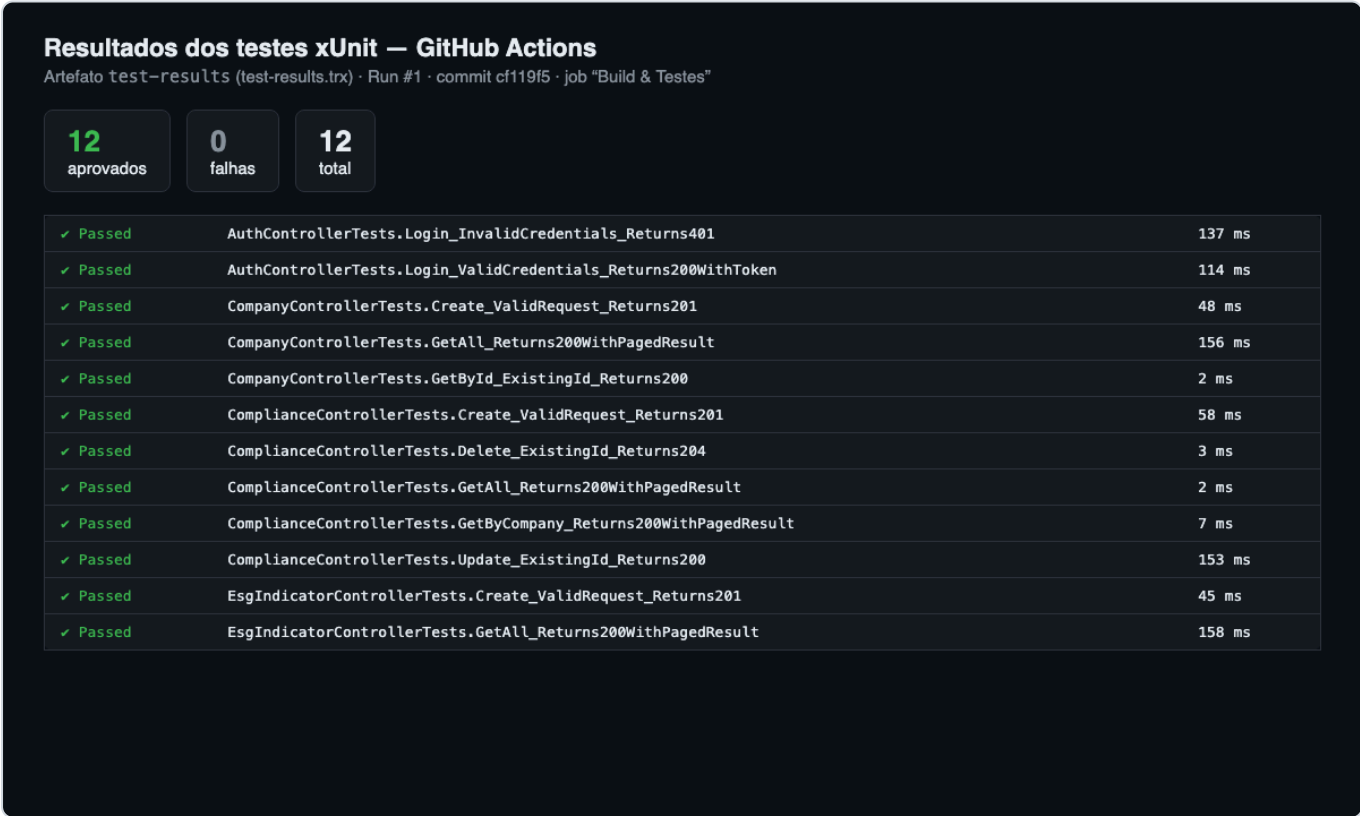


Figura 2: Testes automatizados. Relatório gerado a partir do artefato `test-results.trx` publicado pelo job "Build & Testes": 12 aprovados, 0 falhas.

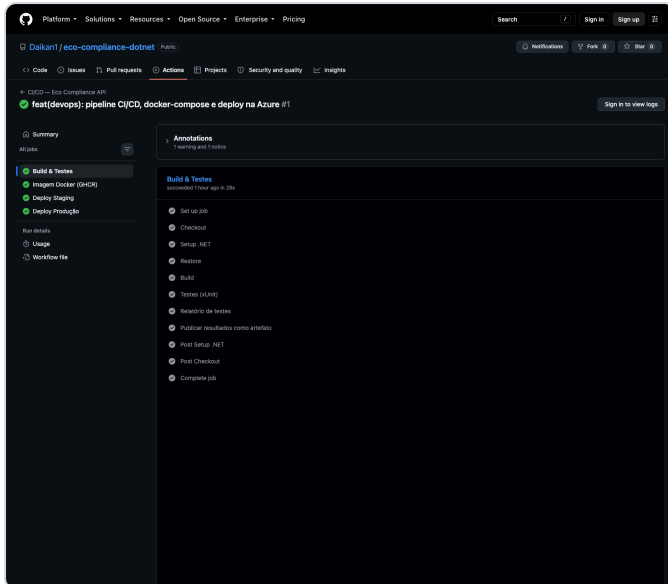


Figura 3: Etapas do job "Build & Testes": checkout, setup .NET, restore, build, testes, relatório e artefato.

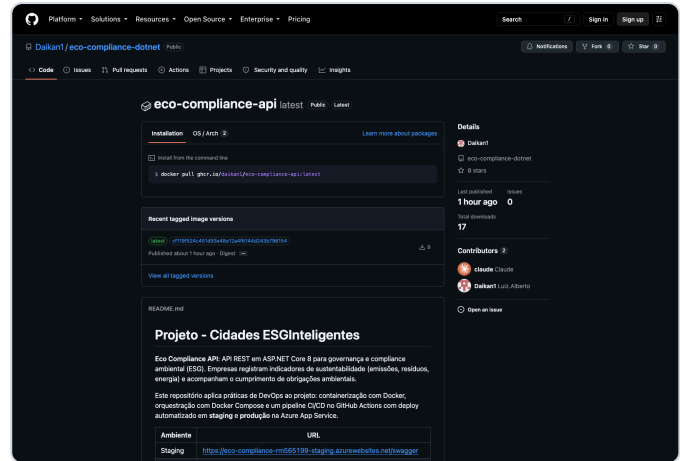


Figura 4: Imagem publicada no GHCR com as tags 'latest' e o SHA do commit.

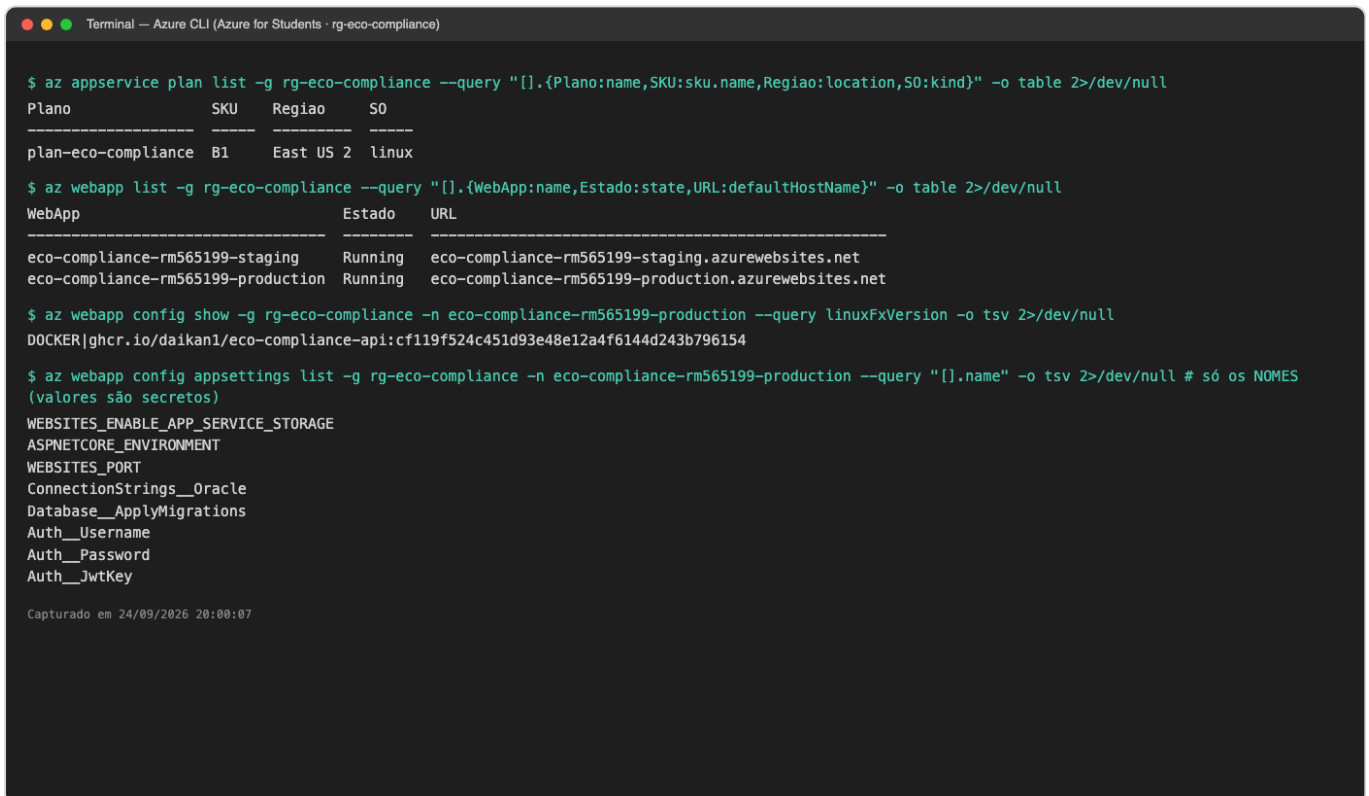


Figura 5: Infraestrutura de deploy na Azure: plano B1 Linux (East US 2), Web Apps de staging e produção em execução e produção rodando a imagem `ghcr.io/daikan1/eco-compliance-api:cf119f5...`. Só os **nomes** das App Settings são exibidos; os valores são secretos.

4. Prints dos ambientes staging e produção

```
Terminal — Ambiente de Staging (Azure App Service)

$ curl -s https://eco-compliance-rm565199-staging.azurewebsites.net/
{"application":"Eco Compliance API - Cidades ESG Inteligentes","environment":"Staging","version":"cf119f524c451d93e48e12a4f6144d243b796154","swagger":"/swagger","health":"/health"}

$ curl -s https://eco-compliance-rm565199-staging.azurewebsites.net/health
Healthy

$ curl -s https://eco-compliance-rm565199-staging.azurewebsites.net/api/companies | python3 -m json.tool --no-ensure-ascii
{
  "items": [
    {
      "id": 2,
      "name": "Secretaria do Verde e Meio Ambiente",
      "cnpj": "46392130000118",
      "sector": "Meio Ambiente"
    },
    {
      "id": 1,
      "name": "Prefeitura de São Paulo",
      "cnpj": "46395000000139",
      "sector": "Governo"
    }
  ],
  "totalCount": 2,
  "page": 1,
  "pageSize": 10,
  "totalPages": 1
}

Capturado em 24/09/2026 20:17:48
```

Figura 6: **Staging**. GET / retorna environment: Staging e o SHA do commit, /health retorna Healthy e /api/companies lista os dados do Oracle.

```
Terminal — Ambiente de Produção (Azure App Service)

$ curl -s https://eco-compliance-rm565199-production.azurewebsites.net/
{"application":"Eco Compliance API - Cidades ESG Inteligentes","environment":"Production","version":"cf119f524c451d93e48e12a4f6144d243b796154","swagger":"/swagger","health":"/health"}

$ curl -s https://eco-compliance-rm565199-production.azurewebsites.net/health
Healthy

$ curl -s https://eco-compliance-rm565199-production.azurewebsites.net/api/companies | python3 -m json.tool --no-ensure-ascii
{
  "items": [
    {
      "id": 2,
      "name": "Secretaria do Verde e Meio Ambiente",
      "cnpj": "46392130000118",
      "sector": "Meio Ambiente"
    },
    {
      "id": 1,
      "name": "Prefeitura de São Paulo",
      "cnpj": "46395000000139",
      "sector": "Governo"
    }
  ],
  "totalCount": 2,
  "page": 1,
  "pageSize": 10,
  "totalPages": 1
}

Capturado em 24/09/2026 20:18:37
```

Figura 7: **Produção**. A mesma versão (cf119f5) foi promovida após aprovação, com environment: Production .

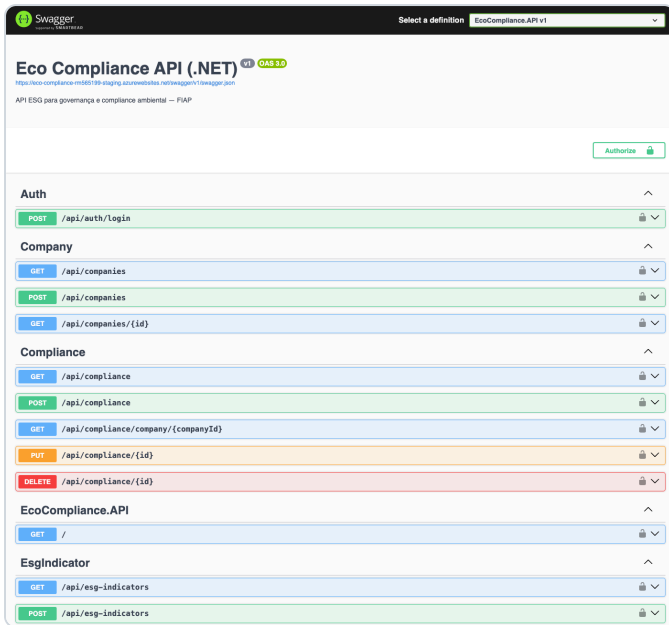


Figura 8: Swagger UI publicado em staging.

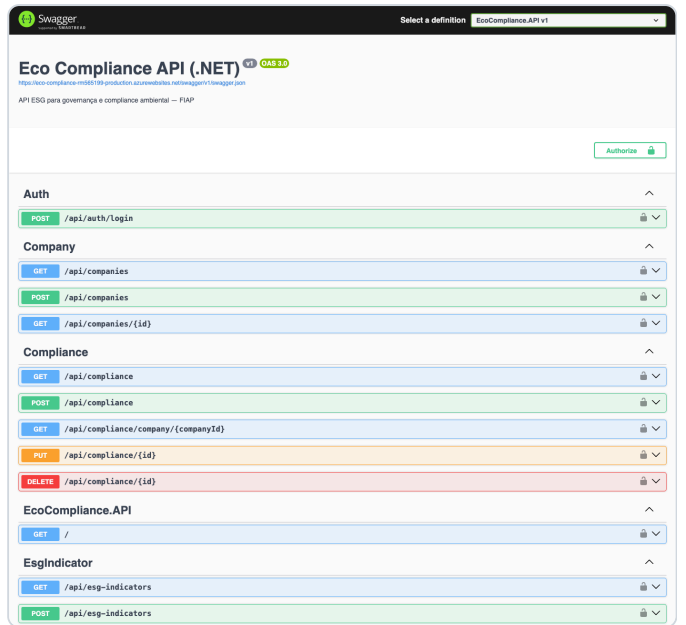


Figura 9: Swagger UI publicado em produção.

```
Terminal — Docker Compose (ambiente local)

$ docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAME          IMAGE                                STATUS      PORTS
eco-api       eco-compliance-api:local           Up 3 hours  0.0.0.0:8095->8080/tcp, [::]:8095->8080/tcp
eco-oracle    gvenzl/oracle-free:23-slim-faststart Up 3 hours (healthy) 0.0.0.0:1521->1521/tcp, [::]:1521->1521/tcp

$ docker volume ls --filter name=oracle-data
DRIVER    VOLUME NAME
local     eco-compliance-api_oracle-data
local     eco-compliance-dotnet_oracle-data

$ docker network ls --filter name=eco-net
NETWORK ID    NAME                                DRIVER    SCOPE
f93b3fbdde0d  eco-compliance-dotnet_eco-net      bridge   local
e063210581f9  eco-net                            bridge   local

$ curl -s localhost:8095/
{"application":"Eco Compliance API - Cidades ESG Inteligentes","environment":"Development","version":"local-compose","swagger":"/swagger","health":"/health"}

$ curl -s localhost:8095/health
Healthy

Capturado em 24/09/2026 19:55:22
```

Figura 10: Ambiente local com Docker Compose. eco-api e eco-oracle (healthy), volume oracle-data, rede eco-net e API respondendo.

5. Desafios encontrados e como foram resolvidos

Desafio	Solução
Tabelas não eram criadas em um banco novo	A migration <code>InitialCreate</code> não tinha os atributos <code>[DbContext]</code> e <code>[Migration]</code> , então o EF Core não a encontrava. Os atributos foram adicionados, junto com a opção <code>Database__ApplyMigrations</code> para aplicar as migrations no startup. O script é idempotente.
Oracle XE sem imagem para Apple Silicon (ARM)	A imagem <code>gvenzl/oracle-xe</code> só existe para amd64 e fica lenta com emulação. Foi trocada por <code>gvenzl/oracle-free:23-slim-faststart</code> , que é multi-arquitetura, com health check para a API só subir com o banco pronto.
Senha real do banco entrava na imagem	O <code>appsettings.Development.json</code> era copiado para dentro da imagem. O <code>.dockerignore</code> passou a excluí-lo, e as credenciais agora entram só por variáveis de ambiente.
Sem evidência visual em staging e produção	O Swagger só era habilitado em Development. Ele foi liberado em todos os ambientes, e foram criados <code>/health</code> e <code>GET / (ambiente + versão)</code> , que também são usados pelo smoke test.
Restrições da assinatura Azure for Students	A região <code>brazilsouth</code> estava bloqueada por política, então foi usada <code>eastus2</code> . O provedor <code>Microsoft.Web</code> não estava registrado e foi registrado com <code>az provider register</code> . O deploy via publish profile exigia habilitar a autenticação básica do SCM, o que foi automatizado no script.
Conta Oracle bloqueada (ORA-28000)	Uma senha digitada incorretamente nas App Settings gerou falhas de login repetidas e bloqueou a conta. A senha foi redefinida com a FIAP, e o script <code>set-db-password.sh</code> foi criado para ler a senha sem eco, pedir confirmação e atualizar os dois ambientes de forma consistente.
Instabilidade do Oracle da FIAP (ORA-50000 timeout)	O smoke test do pipeline valida a aplicação (<code>/health</code> e versão) sem depender do banco externo, então uma instabilidade da FIAP não reprova o deploy. As validações com dados foram repetidas quando o banco estabilizou.
Porta 8080 já ocupada na máquina local	A porta publicada passou a ser configurável no <code>.env</code> (<code>API_PORT</code>), sem alterar o compose.
Git travando na pasta do projeto	A Área de Trabalho estava no iCloud com "Otimizar armazenamento", e os arquivos ficavam só na nuvem. A pasta foi marcada como "Manter Download".

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒

Tecnologias utilizadas

Aplicação	C#, .NET 8, ASP.NET Core Web API, Entity Framework Core 8, Oracle.EntityFrameworkCore, JWT, Swagger
Testes	xUnit, Moq
Containers	Docker (multi-stage), Docker Compose, Oracle Database 23ai Free
CI/CD	GitHub Actions, GitHub Environments (aprovação manual), GitHub Container Registry
Nuvem	Azure App Service (Web App for Containers, Linux), Azure CLI, Oracle FIAP